

SECURE WORKFLOW ARCHITECTURE AND STRIDE THREAT MODEL

DevSecOps Assessment – Multi-Tier Web Application

1. Introduction

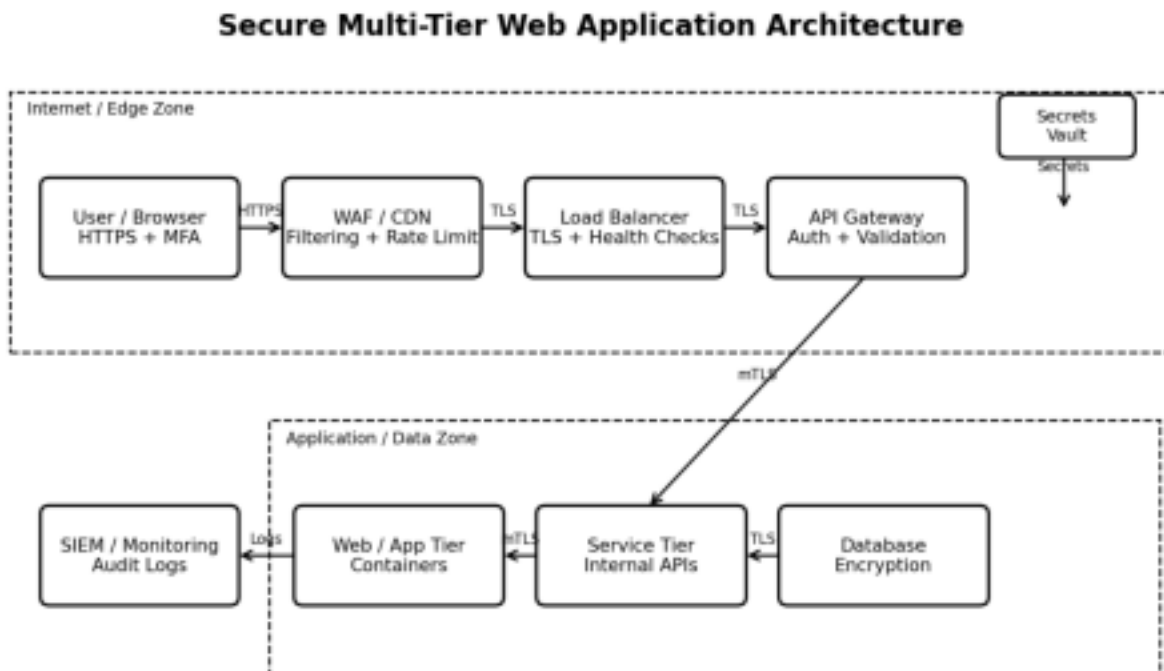
A multi-tier web application separates presentation, application/service, and data processing into logical layers. This assessment designs a secure workflow architecture and applies STRIDE to identify assets, trust boundaries, threats, and security controls throughout the DevSecOps lifecycle.

2. Objectives

- Design a secure three-tier web application architecture.
- Identify critical assets and their security requirements.
- Define trust boundaries and important data flows.
- Apply STRIDE to identify Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege threats.
- Map threats to preventive, detective, and corrective security controls.
- Integrate security checks into the CI/CD pipeline.

3. Secure Multi-Tier Workflow Architecture

The proposed architecture places public traffic behind a WAF/CDN and API gateway, isolates application services from the database, and uses centralized monitoring and secure secrets management.



3.1 Architecture Components

Component	Security Role
-----------	---------------

User / Browser	Initiates requests using HTTPS; MFA can protect authenticated accounts.
----------------	---

DevSecOps Assessment – Secure Workflow Architecture & STRIDE Threat Model

WAF / CDN	Filters malicious web traffic, applies rate limits, bot protection, and TLS controls.
Load Balancer	Distributes traffic to healthy application instances and improves availability.
API Gateway	Centralizes authentication, authorization, validation, throttling, and API routing.
Web / Application Tier	Runs application logic in isolated containers using least-privilege identities.
Service Tier	Provides internal business services with authenticated service-to-service communication.
Database	Stores application data and is reachable only by authorized services.
Secrets Vault	Protects database credentials, API keys, certificates, and other secrets.
SIEM / Monitoring	Collects security logs, detects suspicious activity, and supports incident response.

4. Secure DevSecOps Workflow

Plan: Define security requirements, compliance needs, abuse cases, and threat-model assumptions.

Code: Use secure coding standards, peer review, secret scanning, and pre-commit checks.

Build: Run SAST, software composition analysis, dependency checks, and container image scanning.

Test: Run unit, integration, API, DAST, authentication/authorization, and security regression tests.

Package: Generate an SBOM and sign approved build artifacts and container images.

Deploy: Use infrastructure as code, least privilege, protected environments, and managed secrets.

Operate: Monitor logs, patch vulnerabilities, review configuration, back up data, and investigate alerts.

Respond: Contain incidents, rotate credentials, recover services, and feed lessons back into development.

5. Assets and Security Requirements

Primary Controls

Asset	Need
Credentials / session tokens	C / I
Personal / business data	C / I
Database	C / I / A
Source code	I / C
API keys / secrets	C
Build artifacts / images	I
Logs / audit records	I / A
CI/CD pipeline	I / A
Certificates / signing keys	C / I
Application availability	A

MFA, secure cookies, token expiry, session invalidation Encryption, RBAC, data minimization, access logging Network isolation, least privilege, encryption, backups
Repository access control, branch protection, code review, secret scanning
Secrets vault, rotation, no hard-coded credentials Image scanning, signing, registry access control Centralized collection, restricted access, retention, tamper resistance
Strong authentication, isolated runners, least privilege, approvals
Key management, rotation, restricted access 0 Rate limiting, redundancy, autoscaling, recovery procedures

6. Trust Boundaries

ID	Boundary	Why It Matters	Controls
TB1	Internet ↔ WAF/CDN	Public, untrusted traffic	HTTPS, WAF, rate limiting, DDoS protection
TB2	Edge ↔ API Gateway	Traffic enters controlled application edge	TLS, authenticated routing, request validation
TB3	Gateway ↔ Services	External-originated requests reach internal services	RBAC/ABAC, mTLS, service identities
TB4	Application ↔ Database	Application accesses persistent data	Private network, firewall rules, DB least privilege, TLS
TB5	Application ↔ Secrets Vault	Application retrieves sensitive credentials	Workload identity, short-lived credentials, auditing
TB6	CI/CD ↔ Production	Pipeline can change production systems	Protected branches, signed artifacts, approvals, isolated runners
TB7	Application ↔ SIEM	Security telemetry leaves components	Authenticated log shipping, integrity protection, access control

7. STRIDE Threat Model

STRIDE	Threat Description	Main Controls
S – Spoofing	Impersonation of users, services, administrators, or CI/CD identities	MFA, short-lived tokens, secure sessions, mTLS, workload identity
T – Tampering	Modification of requests, code, artifacts, configuration, or database records	TLS, parameterized queries, signed artifacts, branch protection, integrity checks

DevSecOps Assessment – Secure Workflow Architecture & STRIDE Threat Model

R – Repudiation	Denial of an action because reliable evidence is unavailable	Centralized immutable logs, timestamps, correlation IDs, audit trails
I – Information Disclosure	Exposure of data, credentials, tokens, logs, or source code	RBAC, encryption, secrets vault, log redaction, data minimization
D – Denial of Service	Exhaustion of network, API, application, or database resources	WAF, rate limiting, quotas, caching, autoscaling, circuit breakers
E – Elevation of Privilege	Low-privileged account or service gains higher permissions	Least privilege, RBAC/ABAC,

		server-side authorization, isolation, patching
--	--	--

8. Detailed Threat Scenarios and Mitigations

Threat	STRIDE	Risk
Credential theft	S	High
Session/token manipulation	S/T	High
SQL injection	T/I/E	Critical
Broken access control	E/I	Critical
Malicious dependency	T/E	High
CI/CD compromise	T/E	Critical
Secret leakage	I	High
API abuse / DoS	D	High
Log manipulation	R	Medium

Container escape	E	Critical
Sensitive data exposure	I	High
Key/certificate compromise	S/T	High

9. Security Controls by Layer

Layer	Security Controls
Mitigation	MFA, password hashing, login throttling, anomaly detection
Application	Secure cookies, expiry, signature, key rotation
Database	Parameterized queries, validation, least privileged DB account, SAST/DAST
Server-side	authorization, RBAC/AB, object-level checks
Container	SCA, dependency pinning, SBOM, registries, artifact verification, protected branches, isolated runners, signed artifacts, approvals
Secrets	Secret scanning, vault, redaction, rotation and revocation
WAF	WAF, rate limits, quotas, caching, relimits
Logging	Centralized logging, immutable storage, restricted log access
Images	Minimal images, non-root containers, seccomp/AppArmor, runtime monitoring
Encryption	Encryption, filtering, redaction, access control, secure backups
KMS/vault	KMS/vault, least privilege, rotation, signed certificates

Identity MFA, SSO/OIDC, strong password hashing, service identities, short-lived tokens
Network TLS, private subnets, firewalls/security groups, segmentation, mTLS
Application Input validation, output encoding, CSRF protection, secure headers, authorization checks
Database Least privilege, encryption at rest, TLS, parameterized queries, auditing, backups
Secrets Vault/KMS, automatic rotation, workload identity, secret scanning
CI/CD SAST, SCA, secret scanning, DAST, container scanning, SBOM, signed artifacts, approvals
Runtime Non-root containers, minimal images, resource limits, runtime monitoring, patch management
Monitoring Centralized logs, SIEM, alerting, correlation IDs, audit trails
Recovery Immutable backups, tested restoration, incident-response playbooks, credential revocation

10. Risk Prioritization

Risk	Examples	Treatment
Critical	SQL injection; Broken access control; CI/CD compromise; Container escape	Immediate remediation before production release
High	Credential theft; Token manipulation; Malicious dependency; Secret leakage; API DoS; Data exposure; Key compromise	Remediate in current sprint/release and automate controls
Medium	Log manipulation	Mitigate through centralized logging and monitoring

11. Conclusion

The secure architecture reduces attack surface through segmentation, strong identity controls, encrypted

communication, least privilege, secure secrets handling, and continuous monitoring. STRIDE provides a structured method for identifying threats at trust boundaries and mapping them to practical controls. Integrating SAST, SCA,

DevSecOps Assessment – Secure Workflow Architecture & STRIDE Threat Model

secret scanning, DAST, container scanning, SBOM generation, artifact signing, and security gates into CI/CD makes security a continuous DevSecOps activity.

12. Recommended Security Checklist

- Use HTTPS everywhere and mTLS for sensitive internal service-to-service communication. •
- Enforce MFA for users and privileged administrators.
- Apply least privilege to users, services, databases, cloud resources, and CI/CD identities. •
- Never store passwords, API keys, or private keys in source code.
- Run SAST, SCA, secret scanning, DAST, and container scanning in the pipeline.
- Generate and retain an SBOM for production artifacts.
- Sign and verify container images and deployment artifacts.
- Use centralized, tamper-resistant logging and SIEM monitoring.
- Encrypt sensitive data at rest and in transit.
- Review the threat model whenever architecture or major features change.
- Maintain tested backups and an incident-response process.
- Patch operating systems, frameworks, containers, and dependencies according to risk.

